

DESENVOLVIMENTO DE SOFTWARE PARA A WEB 1

Sistema Web: Marketplace de Produtos (lojas/vendedores + clientes)

Arquitetura: Modelo-Visão-Controlador (MVC)

Atividade AA-1: Marketplace de produtos

O sistema deve possuir um cadastro de **clientes**, com os seguintes dados: e-mail, senha, CPF, nome, telefone, sexo e data de nascimento.

O sistema deve possuir um cadastro de **lojas/vendedores**, com os seguintes dados: e-mail, senha, CNPJ, nome e descrição.

O sistema deve possuir um **administrador** (operador da plataforma), responsável pela gestão de clientes e lojas, cujo login + senha são populados no banco na inicialização.²

Os produtos são organizados em **categorias** e **cada produto pertence a uma loja**. Um **cliente pode comprar produtos de diferentes lojas** num mesmo pedido; cada pedido é composto por um ou mais itens (produto + quantidade).

O sistema deve atender aos seguintes requisitos:

- **R1:** CRUD¹ de **clientes** (requer login de administrador).
- **R2:** CRUD de **lojas/vendedores** (requer login de administrador). Além disso, uma loja pode se **auto-cadastrar** publicamente para começar a vender.
- **R3:** Cadastro de **produtos para venda** (requer login da loja). Depois de fazer login, a loja pode cadastrar, editar e remover **os seus próprios** produtos. Dados do produto: nome, descrição, preço, quantidade em estoque, categoria e fotos (no máximo 10 imagens).
- **R4:** Listagem de todos os produtos em uma única página (não requer login), com filtro por nome, por **loja**, por categoria ou por faixa de preço.
- **R5:** Realização de **pedido de compra** (requer login do cliente). O cliente adiciona ao carrinho produtos de **uma ou mais lojas** e finaliza a compra; a data é registrada, o pedido nasce ABERTO e o cliente é informado por e-mail.
- **R6:** Listagem de todos os **pedidos de um cliente** (requer login do cliente), com os status (ABERTO, PAGO, ENVIADO, CANCELADO) e o valor total.
- **R7:** Gestão das **vendas pela loja** (requer login da loja): a loja visualiza os pedidos que contêm os seus produtos e atualiza o status, notificando o cliente por e-mail. O administrador também visualiza e atualiza o status de todos os pedidos.
- **R8:** O sistema deve ser **internacionalizado** em pelo menos dois idiomas: português + inglês.
- **R9:** O sistema deve **validar** todas as informações (campos nos formulários) cadastradas/editadas.

O sistema deve **tratar todos os erros possíveis** (cadastros duplicados, problemas técnicos, etc.) mostrando uma página de erros amigável ao usuário e registrando o erro no console.

Arquitetura

Modelo-Visão-Controlador (MVC).

Tecnologias

- Spring MVC, Spring Data JPA, Spring Security & Thymeleaf (Lado Servidor)
- JavaScript & CSS (Lado Cliente)

Ambiente de Desenvolvimento

- A compilação e o *deployment* devem ser obrigatoriamente realizados via **Maven**.
- Os arquivos-fonte devem estar hospedados obrigatoriamente em um repositório (**GitHub**).

1. CRUD: Create, Read, Update & Delete.

2. O login + senha do administrador são populados no banco durante a inicialização do sistema.

DESENVOLVIMENTO DE SOFTWARE PARA A WEB 1

Sistema Web: Marketplace de Produtos

Atividade AA-2: Marketplace de produtos — REST API

Obs 1: Esta atividade é baseada na AA-1; implementam-se apenas os novos requisitos (funcionalidades providas em uma REST API).

REST API — CRUD de clientes

- **Cria um novo cliente [Create]**
POST <http://localhost:8080/api/clientes>
Body: raw/JSON (application/json)
- **Lista os clientes [Read]**
GET <http://localhost:8080/api/clientes>
- **Cliente de id = {id} [Read]**
GET <http://localhost:8080/api/clientes/{id}>
- **Atualiza o cliente {id} [Update]**
PUT <http://localhost:8080/api/clientes/{id}>
Body: raw/JSON (application/json)
- **Remove o cliente {id} [Delete]**
DELETE <http://localhost:8080/api/clientes/{id}>

REST API — CRUD de lojas

- **Cria uma nova loja [Create]**
POST <http://localhost:8080/api/lojas>
Body: raw/JSON (application/json)
- **Lista as lojas [Read]**
GET <http://localhost:8080/api/lojas>
- **Loja de id = {id} [Read]**
GET <http://localhost:8080/api/lojas/{id}>
- **Atualiza a loja {id} [Update]**
PUT <http://localhost:8080/api/lojas/{id}>
Body: raw/JSON (application/json)
- **Remove a loja {id} [Delete]**
DELETE <http://localhost:8080/api/lojas/{id}>

REST API — CRUD de produtos (informe categoria e loja pelo id)

- **Cria um novo produto [Create]**
POST <http://localhost:8080/api/produtos>
Body: raw/JSON (application/json)

- **Lista os produtos [Read]**
GET <http://localhost:8080/api/produtos>
- **Produto de id = {id} [Read]**
GET <http://localhost:8080/api/produtos/{id}>
- **Atualiza o produto {id} [Update]**
PUT <http://localhost:8080/api/produtos/{id}>
Body: raw/JSON (application/json)
- **Remove o produto {id} [Delete]**
DELETE <http://localhost:8080/api/produtos/{id}>

REST API — Consultas adicionais

- **Produtos cujo nome contém {nome} [Read]**
GET <http://localhost:8080/api/produtos?nome={nome}>
- **Produtos da categoria {id} [Read]**
GET <http://localhost:8080/api/produtos/categorias/{id}>
- **Produtos da loja {id} [Read]**
GET <http://localhost:8080/api/produtos/lojas/{id}>
- **Cria pedido para o cliente {id} [Create]**
POST <http://localhost:8080/api/pedidos/clientes/{id}>
Body: raw/JSON (application/json)
- **Pedidos do cliente {id} [Read]**
GET <http://localhost:8080/api/pedidos/clientes/{id}>
- **Pedidos por status {status} [Read]**
GET <http://localhost:8080/api/pedidos/status/{status}>

Obs 2: Em todas as funcionalidades da REST API, não há necessidade de autenticação.

```
// Exemplo de corpo para criar um produto (POST /api/produtos)
{
  "nome": "Mouse Gamer",
  "descricao": "RGB",
  "preco": 150.0,
  "estoque": 20,
  "categoria": { "id": 3 },
  "loja": { "id": 1 }
}
```

Arquitetura / Tecnologias / Ambiente

Modelo-Visão-Controlador; Spring MVC (Controladores REST), Spring Data JPA, Spring Security & Thymeleaf; compilação e *deployment* via **Maven**; código no **GitHub**.